

UNIVERSIDAD INDUSTRIAL DE SANTANDER
FACULTAD DE INGENIERÍAS FISICOMECAÑICAS
ESCUELA DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

PLAN DE TRABAJO DE GRADO

FECHA DE PRESENTACIÓN: Bucaramanga, 7 de octubre de 2025.

TÍTULO: Reingeniería del sistema de gestión de menús y permisos de la plataforma Comunidad Académica (COMA)

MODALIDAD: Práctica empresarial

AUTOR: Firma: _____
Nestór Javier Clavijo Hernández - 2211629

AUTOR: Firma: _____
Jesús David Ramírez Celis - 2211593

DIRECTOR: Firma: _____
Luis Ignacio González Ramírez
Docente Escuela de ingeniería de Sistemas

ENTIDAD INTERESADA: Universidad Industrial de Santander

COMITÉ DE TRABAJOS DE GRADO:

EVALUADOR ASIGNADO:

CONCEPTO DEL EVALUADOR:

APROBACIÓN DEL COMITÉ:

FECHA: _____

Acta No. _____

TABLA DE CONTENIDO

1. PLANTEAMIENTO Y JUSTIFICACIÓN DEL PROBLEMA

2. OBJETIVOS

2.1 OBJETIVO GENERAL

2.2 OBJETIVOS ESPECÍFICOS

3. MARCO TEÓRICO

3.1 Reingeniería de software

3.2 Ingeniería inversa

3.3 Reestructuración del código

3.4 Refactorización

3.5 Optimización de código

3.6 Reestructuración de datos

3.7 Arquitectura cliente/servidor

3.8 Java

3.9 Servlets

3.10 JSP (Java Server Pages)

3.11 MySQL

3.12 Apache Tomcat

3.13 Puntero

3.14 Middleware

3.15 Caché

4. METODOLOGÍA

4.1 Fase 1: Análisis del estado actual de la aplicación

4.2 Fase 2: Rediseño de la rutina de inicio de sesión mediante middleware

4.3 Fase 3: Desarrollo del gestor de menús, servicios y permisos

4.4 Fase 4: Identificación y registro de los servicios existentes

4.5 Fase 5: Reestructuración de la base de datos

4.6 Fase 6: Evaluación posterior a la implementación

5. CRONOGRAMA

6. PRESUPUESTO

7. BIBLIOGRAFÍA

1. PLANTEAMIENTO Y JUSTIFICACIÓN DEL PROBLEMA

La plataforma Comunidad Académica (COMA), desarrollada y administrada por el grupo Calumet de la Universidad Industrial de Santander, complementa los espacios de interacción de las diferentes escuelas y apoya el cumplimiento de la misión universitaria en docencia, investigación y extensión. A través de ella, la comunidad puede actualizar información personal, publicar noticias, organizar eventos, gestionar agendas, intercambiar documentos y, en general, comunicarse de manera más eficiente. Dada su criticidad operativa, su arquitectura debe garantizar eficiencia, mantenibilidad y un modelo de seguridad robusto.

En primer lugar, el problema central es el consumo de memoria derivado del manejo de menús. En COMA, los menús son la estructura de navegación que se organiza en forma de árbol, estos contienen enlaces a los servicios de la plataforma; cada servicio corresponde a una funcionalidad o a un punto de acceso a otras funciones. En el diseño vigente, cuando un usuario accede, la plataforma construye una matriz extensa con todos los servicios habilitados para ese tipo de usuario y la almacena íntegramente en su sesión dentro del servidor para reutilizarla al generar el menú. Esta práctica duplica la misma estructura tantas veces como sesiones activas haya, lo que incrementa de manera sostenida la memoria ocupada y tensiona los recursos del servidor. Como efecto secundario, se generan consultas redundantes a la base de datos y se añade latencia en la construcción y uso del menú, lo que agrava la carga del servidor. Además, la incorporación continua de nuevos servicios incrementa el tamaño de la matriz, de modo que cada copia residente en sesión ocupa más memoria; en consecuencia, el consumo total crece tanto por la concurrencia de usuarios como por la expansión funcional de la plataforma con el paso del tiempo.

La propuesta es introducir un gestor central de menús que actúe como caché a nivel de aplicación y que se organice por tipos de usuario (estudiante, egresado, planta, etc.), porque en la práctica los usuarios que comparten un mismo tipo acceden a los mismos menús. De este modo, el gestor crea una única instancia para cada tipo de usuario, por ejemplo, una para el menú de Estudiante, otra para el de Docente y así sucesivamente; cada sesión no almacena la estructura completa, sino únicamente un puntero a la instancia que le corresponda. Por puntero se entiende un identificador que señala dónde está guardada la instancia en el gestor sin copiarla. Con ello se mantiene el uso de la sesión como referencia, se evita la multiplicación de estructuras idénticas entre miles de usuarios y se preserva un acceso rápido a los menús, al tiempo que disminuye de forma significativa el consumo de memoria y la necesidad de consultas redundantes a la base de datos.

En segundo lugar, el sistema presenta un problema de mantenibilidad a nivel de código por la dispersión de la lógica de autorización. Actualmente, el acceso se controla incluyendo manualmente un fragmento de verificación en cada página protegida. Este patrón, repetido en más de mil archivos, eleva el costo de cambio y aumenta el riesgo de omisiones. La propuesta es centralizar la autenticación y la autorización mediante un middleware, entendido como una capa intermedia que recibe cada solicitud, valida la identidad y los permisos del usuario y solo entonces permite que la petición continúe hacia el servicio solicitado. Al ubicar la verificación en un único punto del flujo de solicitudes, se elimina la repetición de código, se reducen las inconsistencias y se facilita la evolución de la plataforma.

No obstante, implementar esta solución conlleva un reto principalmente operativo. Para consolidar el control de acceso en el middleware, será necesario modificar los más de mil archivos que hoy contienen la lógica incrustada. En muchos casos, esa verificación no se incluyó como componente reutilizable, sino que se copió y pegó dentro de cada servicio, lo que exige una depuración cuidadosa, caso por caso, para retirar duplicaciones sin alterar el comportamiento funcional. Una vez completada esta limpieza, las páginas dejarán de contener validaciones propias y el control de acceso quedará concentrado en la capa intermedia, con beneficios directos en coherencia y mantenimiento.

En tercer lugar, el modelo de seguridad actual acopla permisos y menús, lo que introduce vulnerabilidades, ya que proteger un servicio exige incorporarlo a un menú, de modo que existen rutas a servicios no visibles en la navegación. Esa dependencia de la estructura de menú obliga a controles contextuales frágiles que no se apoyan en una definición uniforme de qué se permite y a quién. Entonces, la solución propuesta combina dos acciones coordinadas: separar navegación y autorización en la base de datos, de manera que los menús se conserven solo como mecanismo de visibilidad, y crear un sistema a nivel de aplicación que, apoyado en esa nueva estructura, valide los permisos por servicio de forma eficiente y consistente. Este sistema consultaría una única fuente sobre servicios y tipos de usuarios, pudiendo reutilizar decisiones en caché y aplicando una política de acceso uniforme con independencia de la organización del menú. Con esta separación, cualquier solicitud a una ruta no registrada o sin permiso es detenida por el middleware, incluso si la URL y los parámetros necesarios se conocen.

Para hacer viable esta propuesta, uno de los desafíos más relevantes es construir un inventario completo y confiable de los servicios para registrarlo en la base de datos. Esto implica localizar todas las rutas operativas, incluidas variantes con parámetros, alias y puntos de entrada indirectos; decidir la granularidad con la que se definirá cada servicio; y asignar sus reglas por tipo de usuario. Para lograrlo, se requiere combinar análisis del código con revisión de registros de acceso, pruebas exploratorias y validación con responsables funcionales. Durante la transición se busca aplicar una política de denegación por defecto de servicios, la creación controlada de excepciones temporales y un monitoreo cercano de errores, hasta completar el registro y las pruebas de regresión.

En conjunto, esta reingeniería para la integración de las medidas no solo suma beneficios, sino que establece una dependencia positiva entre ellas. La separación entre menús y permisos crea un modelo único de autorización por servicio que el middleware consulta de forma consistente, sin depender de cómo esté organizado el menú. El gestor central de menús, organizado por tipos de usuario, provee a esa verificación un acceso rápido a la información necesaria y evita reconstrucciones repetidas, porque cada sesión guarda solo un puntero a la instancia de menú que le corresponde. Así, el middleware aplica las reglas sobre una misma fuente, el gestor reduce la carga de datos que esa aplicación requiere y la independencia de los menús garantiza que ninguna ruta quede accesible por fuera de la política central. El resultado es coherencia en las decisiones de acceso, menor latencia bajo carga y una plataforma más sencilla de auditar y mantener.

2. OBJETIVOS

2.1 OBJETIVO GENERAL

Evaluar y rediseñar el sistema de gestión de menús y permisos de la plataforma Comunidad Académica (COMA) de la Universidad Industrial de Santander, centralizando el almacenamiento de los menús en el servidor para optimizar el uso de memoria en las sesiones de los usuarios, al tiempo que se separa la lógica de menús y permisos según los diferentes roles, con el fin de mejorar la gestión de memoria, tiempos de carga y reforzar la seguridad en el control de accesos.

2.2 OBJETIVOS ESPECÍFICOS

2.2.1 Evaluar el estado actual del sistema de gestión de menús y permisos de la plataforma COMA, con el fin de identificar falencias, problemas de seguridad y deficiencias en el uso de memoria.

2.2.2 Diseñar una arquitectura centralizada para la gestión de menús y permisos que elimine la duplicación de código, facilite el mantenimiento y permita un desarrollo más organizado y escalable.

2.2.3 Rediseñar el sistema de gestión de menús a partir de un mecanismo centralizado de almacenamiento en memoria, que permita cargar los menús una única vez y optimice el uso de recursos.

2.2.4 Separar la lógica de control de permisos en el código y en la base de datos, de manera que los permisos no dependan directamente de los menús y se fortalezca la seguridad en el control de accesos.

2.2.5 Eliminar procesos redundantes y operaciones innecesarias presentes en la creación y consulta de menús, con el fin de reducir la carga sobre la base de datos y mejorar el rendimiento del sistema.

3. MARCO DE REFERENCIA

3.1 Reingeniería de software

Según Valenti (2002), la reingeniería de software es un proceso estratégico orientado a renovar y optimizar los sistemas heredados (*legacy systems*) para mantener su utilidad y rentabilidad en contextos tecnológicos cambiantes. A diferencia del mantenimiento tradicional, que se limita a corregir errores o realizar ajustes menores, la reingeniería busca una transformación estructural que mejore la calidad, confiabilidad y mantenibilidad del software.

Entre sus principales ventajas, permite reducir los costos de mantenimiento y aprovechar nuevas plataformas tecnológicas, migrando los sistemas obsoletos hacia entornos más eficientes. Además, posibilita añadir nuevas funcionalidades, lo que incrementa la competitividad del sistema, y preserva el conocimiento corporativo contenido en reglas de negocio, decisiones de diseño y datos críticos.

La reingeniería también disminuye los riesgos de interrupción de servicios, al facilitar una transición controlada frente a la sustitución completa de los sistemas. En este sentido, se convierte en un factor clave para la competitividad y supervivencia organizacional, pues ignorar el problema de los sistemas heredados puede comprometer la estabilidad empresarial. Finalmente, Valenti (2002) resalta que la reingeniería ofrece una visión estratégica de futuro, ya que todo sistema actual puede volverse obsoleto con el tiempo; anticiparse a este proceso constituye una forma de asegurar la evolución y continuidad del software.

3.2 Ingeniería inversa

El concepto de ingeniería inversa tiene su origen en el ámbito del hardware, donde tradicionalmente se emplea para analizar y desensamblar productos desarrollados por la competencia con el propósito de obtener conocimiento empírico sobre su funcionamiento interno, especialmente cuando se desconocen las especificaciones de diseño o los procesos de fabricación. Esta práctica permite comprender la estructura, los componentes y las relaciones funcionales del sistema original a partir de su producto final.

En el contexto del software, la ingeniería inversa se define como el proceso de análisis de un programa existente con el objetivo de obtener una representación de mayor nivel de abstracción que el código fuente. En otras palabras, busca recuperar y comprender el diseño original del sistema, identificando su arquitectura, componentes y flujos de información.

A través de las herramientas y técnicas propias de la ingeniería inversa, es posible extraer información relevante del software existente, incluyendo aspectos del diseño arquitectónico, la lógica de los procesos y la estructura de los datos. Este proceso no solo facilita el mantenimiento y la documentación de sistemas heredados, sino que también constituye una etapa fundamental dentro de

los procesos de reingeniería de software, al proporcionar una base sólida para su posterior modernización y optimización.

3.3 Reestructuración del código

Dentro del proceso de reingeniería de software, una de las actividades más frecuentes y relevantes es la reestructuración del código. Esta consiste en analizar el código fuente existente con el fin de identificar problemas que dificulten su comprensión, así como obstáculos que afecten las tareas de prueba, mantenimiento y evolución del sistema. A partir de dicho análisis, se aplican técnicas orientadas a mejorar la organización y la legibilidad del código, manteniendo inalterada su funcionalidad original.

El proceso de reestructuración implica refactorizar y optimizar las estructuras de programación, eliminando redundancias, corrigiendo malas prácticas y adaptando el código a estándares modernos de calidad. Una vez realizado este proceso, el código resultante se evalúa según criterios de calidad establecidos, garantizando que cumpla con los requisitos funcionales y no presente anomalías. Finalmente, se procede a actualizar la documentación interna del sistema, asegurando la coherencia entre el código y su descripción técnica, lo cual facilita su posterior mantenimiento y comprensión por parte de los desarrolladores.

3.4 Refactorización

La refactorización es una práctica esencial dentro de la reingeniería de software que consiste en modificar la estructura interna del código sin alterar su comportamiento externo. Su propósito principal es mejorar la claridad, la legibilidad y la mantenibilidad del software, facilitando futuras tareas de desarrollo y reduciendo la probabilidad de errores.

Durante el proceso de refactorización, se aplican técnicas sistemáticas de reorganización y simplificación del código, como la eliminación de duplicaciones, la mejora de nombres de variables y métodos, y la reestructuración de bloques lógicos o funciones. Estas acciones buscan obtener un código más coherente, modular y fácil de entender, lo que contribuye directamente a una mayor calidad del producto software.

La refactorización no pretende añadir nuevas funcionalidades, sino perfeccionar la base existente para hacerla más robusta y adaptable a cambios futuros. Una vez completado el proceso, se realiza una evaluación del código y pruebas de regresión para garantizar que el comportamiento funcional del sistema se mantenga inalterado. De esta forma, la refactorización actúa como un puente entre el mantenimiento y la evolución continua del software.

3.5 Optimización de código

La optimización de código es una actividad orientada a mejorar el rendimiento y la eficiencia de un sistema software mediante la revisión y ajuste de su código fuente. A diferencia de la refactorización, cuyo enfoque principal es estructural, la optimización busca reducir el consumo de recursos, mejorar los tiempos de ejecución y optimizar el uso de memoria y procesamiento.

Este proceso implica analizar el comportamiento del sistema para identificar cuellos de botella, operaciones redundantes o estructuras ineficientes. Posteriormente, se aplican técnicas de optimización como la mejora de algoritmos, la reducción de complejidad computacional o el uso eficiente de estructuras de datos. El objetivo es obtener un código más rápido y liviano, sin comprometer su funcionalidad ni su legibilidad.

3.6 Reestructuración de datos

La reestructuración de datos constituye una etapa fundamental dentro de los procesos de reingeniería de software, orientada a mejorar la organización, consistencia y eficiencia de la arquitectura de datos de un sistema. En la mayoría de los casos, esta actividad inicia con un proceso de ingeniería inversa, mediante el cual se analiza detalladamente la estructura de datos existente para comprender su diseño actual y las relaciones entre sus componentes.

Durante esta fase, se definen los modelos de datos necesarios, se identifican las entidades, atributos y sus interrelaciones, y se evalúa la calidad de las estructuras presentes. Cuando se determina que la arquitectura de datos es ineficiente o limitada, se procede a aplicar una reingeniería de datos con el fin de optimizar su diseño y adecuarlo a los estándares modernos de gestión de información.

Dado que la arquitectura de datos influye directamente en la arquitectura del software y en los algoritmos que este emplea, los cambios realizados en las estructuras de datos pueden requerir ajustes significativos tanto en el código fuente como en la organización general del sistema. En consecuencia, la reestructuración de datos no solo mejora el rendimiento y la integridad de la información, sino que también contribuye a la estabilidad, escalabilidad y mantenibilidad global del software, garantizando su alineación con los objetivos evolutivos de la reingeniería.

3.7 Arquitectura cliente/servidor

La arquitectura cliente/servidor es un modelo de diseño en el que las tareas del sistema se dividen entre dos entidades principales: el cliente, que solicita servicios, y el servidor, que los proporciona. En este esquema, el cliente envía peticiones al servidor, el cual las procesa y devuelve las respuestas correspondientes.

La mayor carga de procesamiento recae sobre el servidor, encargado de gestionar bases de datos y ejecutar la lógica de la aplicación, mientras que el cliente se enfoca en la interacción con el usuario y la presentación de resultados.

3.8 Java

Java es un lenguaje de programación de propósito general, orientado a objetos y desarrollado por Sun Microsystems a mediados de la década de 1990, actualmente mantenido por Oracle Corporation. Su principal característica es la portabilidad, ya que los programas escritos en Java pueden ejecutarse en cualquier plataforma que disponga de una Máquina Virtual de Java (JVM), bajo el principio de *“write once, run anywhere”* (escribir una vez, ejecutar en cualquier lugar).

Gracias a su arquitectura multiplataforma, su enfoque modular y su amplia comunidad de desarrollo, Java se ha consolidado como uno de los lenguajes más utilizados en la industria del software. Su versatilidad lo convierte en una herramienta fundamental para el desarrollo de aplicaciones cliente/servidor, sistemas web y proyectos orientados a la reingeniería de software.

3.9 Servlets

Los Servlets son componentes del lado del servidor desarrollados en Java que permiten generar contenido web dinámico y gestionar la comunicación entre los clientes y las aplicaciones del servidor. Actúan como intermediarios que procesan las solicitudes HTTP enviadas por los navegadores, ejecutan la lógica de negocio correspondiente y devuelven las respuestas apropiadas, generalmente en formato HTML o JSON.

Un servlet se ejecuta dentro de un contenedor web, como Apache Tomcat, que se encarga de su administración, seguridad y ciclo de vida. Entre sus principales ventajas destacan su eficiencia, portabilidad y reutilización de código, así como su integración con otras tecnologías Java, como JavaServer Pages (JSP) y Java Database Connectivity (JDBC).

3.10 JSP (Java Server Pages)

JavaServer Pages (JSP) es una tecnología desarrollada por Sun Microsystems que permite la creación de páginas web dinámicas mediante la integración de código Java dentro de contenido HTML. JSP forma parte del entorno Java EE (Enterprise Edition) y se utiliza principalmente para generar interfaces web que interactúan con aplicaciones del lado del servidor.

Esta tecnología se basa en el modelo cliente/servidor, donde las páginas JSP son procesadas por un servidor de aplicaciones, el cual traduce las etiquetas y el código Java embebido en servlets que generan dinámicamente el contenido solicitado por el usuario.

3.11 MySQL

MySQL es un sistema de gestión de bases de datos relacional (RDBMS) de código abierto, ampliamente utilizado por su rendimiento, estabilidad y facilidad de uso. Emplea el lenguaje SQL (Structured Query Language) para la definición y manipulación de los datos, permitiendo organizar la información en tablas relacionadas entre sí.

Este sistema destaca por su eficiencia en entornos cliente/servidor, su compatibilidad multiplataforma y su integración con diversos lenguajes de programación, entre ellos Java, lo que lo convierte en una opción común para el desarrollo de aplicaciones web y empresariales. Gracias a su arquitectura escalable y segura, MySQL constituye una base sólida para la gestión confiable de datos dentro de proyectos de reingeniería y modernización de software.

3.12 Apache Tomcat

Apache Tomcat es un servidor web y contenedor de servlets desarrollado por la Apache Software Foundation, diseñado para ejecutar aplicaciones web basadas en Java, incluyendo tecnologías como Servlets y JavaServer Pages (JSP). Su función principal es procesar las peticiones de los clientes, ejecutar la lógica del lado del servidor y generar las respuestas dinámicas que se envían al navegador.

Tomcat se caracteriza por su ligereza, estabilidad y facilidad de configuración, lo que lo convierte en una herramienta ideal para el despliegue de aplicaciones web Java. Además, su naturaleza de código abierto y su compatibilidad con estándares Java EE lo hacen una opción ampliamente adoptada en entornos académicos y empresariales que buscan implementar soluciones web seguras y escalables.

3.13 Puntero

En informática, un puntero es una variable que almacena la dirección de memoria de otro dato o elemento. Su función principal es permitir el acceso indirecto a los valores almacenados en memoria, facilitando la manipulación dinámica de estructuras como arreglos, listas enlazadas o árboles. Los punteros son fundamentales en lenguajes de bajo nivel como C y C++, donde ofrecen control directo sobre la memoria, aunque también implican un mayor riesgo de errores, como fugas de memoria o accesos inválidos. En términos conceptuales, los punteros constituyen un mecanismo esencial para optimizar el uso de recursos y la eficiencia en la ejecución de programas.

3.14 Middleware

El middleware es un software intermediario que facilita la comunicación, integración y gestión de datos entre diferentes aplicaciones o sistemas distribuidos. Su función principal es actuar como un puente tecnológico que permite la interoperabilidad entre plataformas, herramientas y bases de datos heterogéneas, garantizando un flujo de información eficiente y transparente. Gracias a esta capa intermedia, es posible unificar diversos componentes en un sistema coherente, ofreciendo a los usuarios una experiencia de servicio integrada sin percibir las diferencias entre los entornos subyacentes.

En el ámbito del desarrollo de software, el middleware desempeña un papel esencial al simplificar los procesos de conexión y coordinación entre los distintos módulos o microservicios que conforman una aplicación moderna. De esta manera, los desarrolladores pueden concentrarse en la lógica de negocio y la innovación funcional, en lugar de invertir tiempo en la creación de mecanismos de comunicación personalizados. En ausencia del middleware, cada componente requeriría un módulo propio de intercambio de datos, lo que aumentaría significativamente la complejidad, los costos de mantenimiento y el riesgo de errores en la integración del sistema.

3.15 Caché

La caché es una tecnología de almacenamiento temporal diseñada para almacenar copias de datos, archivos o recursos utilizados con frecuencia, con el fin de mejorar el rendimiento y la eficiencia en el acceso a la información. Su función principal consiste en reducir el tiempo de carga y la latencia en los sistemas, permitiendo que los datos solicitados con anterioridad se obtengan desde una ubicación local en lugar de descargarse o procesarse nuevamente desde la fuente original.

Este mecanismo no sólo acelera los tiempos de respuesta de aplicaciones y páginas web, sino que también optimiza el uso del ancho de banda y los recursos del sistema, contribuyendo a una experiencia de usuario más fluida. La tecnología de caché se implementa en distintos niveles del ecosistema informático como navegadores, sistemas operativos, servidores o procesadores y desempeña un papel fundamental en la mejora del desempeño de redes y plataformas digitales.

4. METODOLOGÍA

La metodología adoptada para el desarrollo de este proyecto se estructuró en seis fases consecutivas, cada una orientada a dar solución progresiva al problema identificado en la gestión de autenticación, servicios y permisos. A continuación, se describe cada una de ellas:

4.1 Fase 1: Análisis del estado actual de la aplicación

En esta primera etapa se realiza un diagnóstico de la plataforma con el fin de identificar su comportamiento en condiciones reales de uso. Para ello se definen y ejecutan actividades de evaluación que cubren desempeño y seguridad. Con el fin de garantizar repetibilidad y trazabilidad en futuros ciclos de desarrollo o mantenimiento, se prioriza la definición de un plan de pruebas reutilizables.

Actividad 1: Se elaborará un plan de pruebas formal y reutilizable que incluya: propósito y alcance, criterios de entrada y salida, especificación de casos y datos de prueba, guiones de ejecución, criterios de automatización, plantillas de reporte y gestión de incidencias. Este plan servirá como base operativa para las actividades de medición de latencia y consumo de memoria.

Actividad 2: Se evaluará el rendimiento de la aplicación bajo distintos niveles de usuarios concurrentes, siguiendo el plan de pruebas definido en la Actividad 1. Se registran métricas como tiempo medio, así como el comportamiento bajo aumentos graduales de carga.

Actividad 3: Con base en los casos y guiones del plan de pruebas, se medirá el uso de memoria asociado a sesiones de usuario. Se analizará el impacto de la carga sobre los recursos del sistema y la estabilidad durante períodos sostenidos de uso.

Actividad 4: Se revisará la correcta gestión de permisos, roles y controles de acceso, así como la interacción de los usuarios con los servicios disponibles.

4.2 Fase 2: Rediseño de la rutina de inicio de sesión mediante middleware

En esta segunda etapa se busca eliminar la redundancia y la complejidad que actualmente existen en los procesos de autenticación. La estrategia consiste en centralizar la validación de usuarios a través de un middleware unificado, capaz de gestionar el acceso a los diferentes servicios de forma eficiente.

Actividad 5: Se eliminarán las rutinas de inicio de sesión independientes, identificando y retirando los métodos de autenticación propios de cada servicio para centralizar la validación de usuarios en el nuevo componente de autenticación.

Actividad 6: Se unificarán las rutinas en el middleware de autenticación, desarrollando un filtro encargado de centralizar la validación de credenciales, gestionar el mapeo de rutas de acceso y asegurar un flujo homogéneo de inicio de sesión en toda la plataforma.

4.3 Fase 3: Desarrollo del gestor de menús, servicios y permisos

En esta fase se implementa un módulo que proveerá al middleware la información de menús, servicios y permisos mediante un mecanismo de almacenamiento local, con el fin de optimizar la validación de accesos y reducir los tiempos de respuesta.

Actividad 7. Se diseñará e implementará el módulo de gestión de menús, servicios y permisos.

4.4 Fase 4: Identificación y registro de los servicios existentes

Para garantizar que el middleware tenga cobertura sobre todos los recursos de la aplicación, en esta fase se realiza un barrido exhaustivo de los archivos de la plataforma, con el fin de localizar y documentar cada uno de los servicios implementados o invocados.

Actividad 8: Se establecerán los factores que determinarán cuándo un servicio está aislado o no debidamente integrado, definiendo criterios que servirán como referencia para su detección automática en análisis posteriores.

Actividad 9: Se implementará un script, basado en los criterios definidos, que recorrerá los archivos de la plataforma para identificar servicios aislados, estableciendo un mecanismo sistemático y repetible para su detección.

Actividad 10: Se depurará manualmente la lista de servicios generada por el script, corrigiendo errores y recuperando posibles registros que no hayan sido detectados automáticamente.

4.5 Fase 5: Reestructuración de la base de datos

En esta fase se redefine la estructura de la base de datos con el propósito de separar la relación entre menús, servicios y permisos. El objetivo es que cada uno de estos elementos pueda administrarse de forma independiente, evitando dependencias rígidas y permitiendo que el middleware gestione los accesos con mayor flexibilidad y escalabilidad.

Actividad 11: Se separarán los menús y los servicios en tablas independientes para evitar que compartan la misma estructura y permitir su gestión de forma autónoma.

Actividad 12: Se implementarán en el backend las clases necesarias para administrar las nuevas tablas de menús y servicios, garantizando la correcta interacción de la aplicación con la base de datos reorganizada.

Actividad 13: Se estructurará la lógica de la aplicación para que cada componente (menús, servicios y permisos) sea gestionado de manera diferenciada y coherente con la nueva arquitectura de la base de datos.

4.6 Fase 6: Evaluación posterior a la implementación

En esta etapa se realiza un nuevo análisis del estado de la aplicación, replicando las mismas métricas de la fase inicial: latencia de inicio de sesión, consumo de memoria por usuario y validación de la seguridad en los accesos. Estos resultados serán comparados con los obtenidos en la Fase 1, con el fin de establecer las mejoras alcanzadas, cuantificar los beneficios del rediseño e identificar oportunidades de optimización futura.

Actividad 14: Se medirá la latencia de inicio de sesión aplicando los casos, datos y guiones definidos en el Plan de Pruebas Estructurado, con escenarios de concurrencia tras los cambios realizados.

Actividad 15: Se evaluará el consumo de memoria por usuario usando los mismos escenarios y criterios del Plan de Pruebas Estructurado, garantizando repetibilidad y comparabilidad tras los cambios

Actividad 16: Se revisará la correcta gestión de permisos, roles y controles de acceso, así como la interacción de los usuarios con los servicios disponibles tras los cambios implementados.

Actividad 17: Consolidar métricas y analizar resultados conforme al Plan de Pruebas Estructurado, comparándolos con las ejecuciones anteriores del plan para identificar variaciones y documentar conclusiones en las plantillas establecidas.

5. CRONOGRAMA

Actividad	Semanas																	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Fase 1	Análisis del estado actual de la aplicación																	
Actividad 1: Plan de pruebas																		
Actividad 2: Medición de latencia																		
Actividad 3: Evaluación memoria																		
Actividad 4: Análisis de seguridad																		
Fase 2	Rediseño de la rutina de inicio de sesión mediante middleware																	
Actividad 5: Eliminación de rutinas de logueo																		
Actividad 6: Desarrollo del middleware																		
Fase 3	Desarrollo del gestor de menús, servicios y permisos																	
Actividad 7: Desarrollo del módulo de caché																		
Fase 4	Identificación y registro de los servicios existentes																	
Actividad 8: Definición de criterios para los servicios																		
Actividad 9: Creación y uso del script para encontrar todos los servicios																		
Actividad 10: Depuración manual del script																		
Fase 5	Reestructuración de la base de datos																	
Actividad 11: Separar las tablas de servicios y menús en la db																		
Actividad 12: Definición de clases para gestionar las tablas en el backend																		
Actividad 13: Estructuración de la lógica para que la logia de menús y servicios est separada																		
Fase 6	Evaluación posterior a la implementación																	
Actividad 14: Medición de latencia																		
Actividad 15: Evaluación memoria																		
Actividad 16: Análisis de seguridad																		
Actividad 17: Análisis de resultados																		

6. PRESUPUESTO

La estimación de gastos para la ejecución del proyecto durante el periodo estipulado en el cronograma se presenta a continuación. La unidad monetaria empleada para este presupuesto es el peso colombiano (COP). La financiación de todos los recursos necesarios están cubiertos por la Universidad Industrial de Santander (UIS) y recursos propios.

6.1 Recursos Humanos

NOMBRE	FUNCION	DEDICACIÓN [HRS/SEMANA]	COSTO POR HORA	TOTAL	PATROCINADOR
Magister Luis Ignacio González Ramírez	Director	4	\$131,000	\$8,384,000	UIS
Nestór Javier Clavijo Hernández	Autor	20	\$11,863	\$3,796,160	UIS
Jesús David Ramírez Celis	Autor	20	\$11,863	\$3,796,160	UIS
Subtotal:				\$15,976,320	

6.2 Equipos e infraestructura

ELEMENTO	Unidad	COSTO	COSTO TOTAL	PATROCINADOR
Servicio eléctrico	mes	\$300,000	\$1,200,000	UIS
Internet	mes	\$210,000	\$840,000	UIS
Equipo de computo	unidad	\$4,500,000	\$9,000,000	Autores
Subtotal:			\$11,040,000	

6.3 Presupuesto general

ELEMENTO	EFFECTIVO	ESPECIE	PATROCINADOR
Recursos humanos	\$0	\$15,976,320	UIS
Equipos e infraestructura	\$0	\$11,040,000	AUTORES/UIS
Subtotal por tipo:	\$0	\$27,016,320	
Total	\$27,016,320		

7. BIBLIOGRAFÍA

- [1] Pape, T., Bolz, C. F., & Hirschfeld, R. (2016, junio 21). Adaptive Just-in-time Value Class Optimization for Lowering Memory Consumption and Improving Execution Time Performance. arXiv preprint arXiv:1606.06726. <https://arxiv.org/abs/1606.06726>
- [2] Refactoring.Guru. (s.f.). Flyweight [Patrón de diseño]. Recuperado de <https://refactoring.guru/design-patterns/flyweight>
- [3] Escalante Pinilla, J. D., & Pérez Bolívar, D. M. (2025, 27 de febrero). Migración de la plataforma COMA desplegada para cada escuela a un solo servidor y realización de actividades de mantenimiento (Trabajo de grado, Universidad Industrial de Santander). Universidad Industrial de Santander. Recuperado de <https://noesis.uis.edu.co/handle/20.500.14071/45145>
- [4] Malkapuram, S. (2025, 16 de mayo). Secure by Design: Modernizing Authentication With Centralized Access and Adaptive Signals. DZone. Recuperado de <https://dzone.com/articles/centralized-authentication-access>.
- [5] Garrido Abenza, P. P. (2015). *Comenzando a programar con JAVA*. Universidad Miguel Hernández. Recuperado de <https://books.google.com.co/books?hl=es&lr=&id=4v8QCgAAQBAJ&oi=fnd&pg=PP1>.
- [6] Gómez Fuentes, M. del C., & Cervantes Ojeda, J. (2017). *Introducción a la programación web con Java: JSP y Servlets, JavaServer Faces*. Universidad Autónoma Metropolitana, Unidad Cuajimalpa. Recuperado de <http://ilitia.cua.uam.mx:8080/jspui/handle/123456789/996>
- [7] Cobo, Á. (s.f.). *PHP y MySQL: Tecnología para el desarrollo de aplicaciones web*. Ediciones Díaz de Santos. Recuperado de <https://books.google.com.co/books?hl=es&id=zMK3GOMOpQ4C&pg=PR17>.
- [8] Universidad Distrital Francisco José de Caldas — Facultad de Ingeniería (s.f.). *Apache Tomcat* [Descripción en laboratorio]. Recuperado de <https://facingeneria.udistrital.edu.co/laboratorios/labing/index.php/laboratorios/apache-tomcat>
- [9] Valenti, S. (2002). *Successful Software Reengineering* [Libro]. IGI Global. Recuperado de <https://books.google.com.co/books?id=PRpEp9nn2tUC>
- [10] EducaOpen. (s.f.). *¿Qué es un puntero en programación?* [Metaterminos]. Recuperado de <https://www.educaopen.com/digital-lab/metaterminos/p/puntero>
- [11] Amazon Web Services. (s.f.). *What is middleware?* [Definición técnica]. Recuperado de <https://aws.amazon.com/es/what-is/middleware>